

SettleUp SRS — Samajhne Ki Guide

Beginner Hinglish Explanation + Viva Preparation

Companion document to: SettleUp SRS v2.0 **Purpose:** Har section ko simple bhasha mein samajhna, aur viva ke liye tayyari

Isko kaise use karna hai

Yeh document SRS ke saath padhne ke liye hai. SRS **professional** hai — usme formal language hai kyunki woh submit hoga. Yeh guide **samajhne** ke liye hai.

Suggestion: roz 10 minute, ek section. Ek hi baithak mein poora ratne ki koshish mat karna.

Har section mein yeh milega: 1. Section ka matlab 2. Yeh section hota hi kyun hai 3. SettleUp ke liye khaas kya hai 4. Technical shabdon ka matlab 5. Viva ke sawaal aur jawab

PEHLE — SRS Kya Cheez Hai

Full form: Software Requirements Specification

Simple matlab: Software banane se pehle likha jaata hai ki **kya banana hai**.

Ghar wala example: Ghar banane se pehle naksha banta hai — kitne kamre, kahan kitchen, kahan bathroom. Mazdoor naksha dekh ke kaam karta hai. Agar naksha na ho to har banda apni marzi se banayega aur ghar bekaar niklega.

SRS software ka naksha hai.

Zaroorat kyun padi: Bina likhe kaam shuru karo to beech mein pata chalta hai "yeh feature to socha hi nahi tha", aur phir aadha code dobara likhna padta hai.

Viva Sawaal

Q: SRS kya hota hai? A: Software Requirements Specification. Yeh ek document hai jo development se pehle likha jaata hai aur batata hai ki system ko kya-kya karna hai, kaise kaam karna hai, aur kaunsi conditions follow karni hain.

Q: SRS kyun zaroori hai? A: Teen wajah. Ek — sabko clear ho jaata hai ki kya banana hai. Do — beech mein confusion nahi hoti. Teen — baad mein check kar sakte hain ki jo plan kiya tha woh bana ya nahi.

Q: SRS aur project report mein kya farak hai? A: SRS **pehle** likhte hain — batata hai kya banega. Report **baad mein** likhte hain — batati hai kya bana. SRS future tense mein hota hai, report past tense mein.

SECTION 1 — INTRODUCTION

1.1 Purpose

Matlab: Yeh document kis liye likha hai aur kaun padhega.

Kyun hota hai: Padhne wale ko pehle paragraph mein pata chal jaana chahiye ki yeh document uske kaam ka hai ya nahi.

SettleUp mein: Teen log padhenge — teacher, main (developer), aur koi bhi jo aage isko badhaye.

1.2 Scope

Matlab: Project ki **boundary**. System kya karega, aur kya **nahi** karega.

Yaad rakhne ka tareeka: Scope ek line hai jo batati hai andar kya hai, bahar kya hai.

Sabse important baat: "Kya nahi karega" wali list. Zyadatar students yeh nahi likhte, aur yahi sabse zyada kaam aati hai.

Example: SRS mein likha hai ki SettleUp **asli paisa transfer nahi karega**. Agar teacher pooche "isme payment gateway kyun nahi hai" to jawab hai: "Sir, scope mein maine pehle hi define kiya tha ki system sirf calculate karega, transfer nahi. Yeh section 1.2 mein likha hai."

Bina likhe hote to yeh **kami** lagti. Likh dene se yeh **decision** ban gaya.

1.3 Project Objectives

Matlab: Yeh project banane se kya **haasil** hoga.

Scope aur Objective ka farak — yeh viva mein zaroor aayega

	Scope	Objective
Sawaal	"Kya banega?"	"Kyun banega?"
Batata hai	Boundary	Purpose
Example	"System settlement generate karega"	"Members ko kam payments karni padein"

Simple tareeka yaad rakhne ka: Scope = **kaam**. Objective = **fayda**.

Viva Sawaal

Q: Scope aur objective mein kya farak hai? A: Scope batata hai ki project mein kya-kya include hai aur kya nahi — matlab boundary. Objective batata hai ki yeh project banane se kya faayda hoga — matlab goal. Jaise "settlement plan banana" scope hai, aur "logon ko kam payments karni padein" objective hai.

Q: Tumhare project ke objectives kya hain? A: Paanch hain sir. Manual calculation hatana, payments ki sankhya kam karna, bina account aur internet ke chalna, data user ke apne device pe rakhna, aur HTML/CSS/JavaScript mein practical skill dikhana.

1.4 Definitions

Matlab: Document mein jo mushkil shabd use hue, unka matlab.

Kyun hota hai: Taaki har banda ek hi cheez samjhe. Agar "creditor" ka matlab clear likha hai to koi confusion nahi.

SettleUp ke sabse important shabd:

Shabd	Matlab
Creditor	Jise paisa milna hai (balance positive)
Debtor	Jise paisa dena hai (balance negative)
Net balance	Kitna diya – kitna share tha
Settlement	Payments ki list jo sab balance zero kar de

Yaad rakhne ka tareeka: Credit = paisa aayega. Debt = udhaar, matlab dena hai.

SECTION 2 — OVERALL DESCRIPTION

2.1 Product Perspective

Matlab: System akela hai ya kisi bade system ka hissa hai.

SettleUp mein: Bilkul akela. Kisi server, database ya service pe depend nahi karta.

Teen layers — yeh samajhna zaroori hai

SRS mein likha hai ki code teen hisson mein banega:

Layer	Kaam	DOM chhuta hai?	Calculation karta hai?
Presentation	Screen pe dikhana, input lena	Haan	Nahi
Logic	Calculation, validation	Nahi	Haan
Persistence	Save/load karna	Nahi	Nahi

Yeh alag kyun rakhe: Settlement wala calculation sabse zyada galti hone wali cheez hai. Agar woh code DOM se juda hua ho to test karna mushkil ho jaata hai. Alag rakhne se usko akele check kar sakte hain — sirf numbers daalo, numbers nikaalo.

Viva Sawaal

Q: Tumne code ko layers mein kyun divide kiya? A: Sir, calculation wala code alag rakha hai taaki usko screen ke bina test kar sakoon. Settlement algorithm sabse mushkil part hai, aur agar woh DOM code ke saath mixed hota to galti dhoondhna mushkil hota.

2.3 Intended User

Matlab: Kaun use karega.

Kyun hota hai: Interface user ke hisaab se banana padta hai. Technical banda ho to shortcuts challenge. Normal banda ho to sab kuch screen pe likha hona chahiye.

SettleUp mein: Normal log — students, dost. Isliye interface simple hoga, koi jargon nahi.

2.4 Operating Environment

Matlab: System kahan chalega — kaunsa device, kaunsa browser.

SettleUp mein: Koi bhi modern browser, mobile ya laptop. Na installation, na server.

2.5 Design and Implementation Constraints

Matlab: Woh cheezein jo **majboori** mein follow kar rahe ho.

Constraint aur requirement ka farak — important:

- **Requirement** = jo system ko karna hai (system add member kar sakega)
- **Constraint** = jo majboori hai (React use nahi kar sakte)

Constraint tumhari marzi se nahi aata. Kahin bahar se aata hai — college se, technology se, ya situation se.

SettleUp ke main constraints:

Constraint	Kahan se aaya
Sirf HTML/CSS/JS	College requirement
Koi framework nahi	College requirement
Koi backend nahi	Project frontend-only hai
localStorage hi use hoga	Database available hi nahi

Viva Sawaal — yeh teenon bahut common hain

Q: Vanilla JavaScript kyun use kiya, React kyun nahi? A: Do wajah sir. Ek, course requirement hai ki framework use na karein. Do, project ka maqsad hi yeh hai ki DOM, events aur state khud handle karna seekhoo. React yeh sab chhupa deta hai, toh jo seekhna tha wahi nahi seekh paata.

Q: localStorage kyun use kiya, database kyun nahi? A: Sir, project frontend-only hai, isliye koi database server available nahi hai. localStorage browser ka apna feature hai jo data save rakhta hai. Mere case mein data chhota hai aur ek hi user ka hai, isliye yeh kaafi hai. Iski limitations maine section 10.2 mein likhi hain.

Q: Backend kyun nahi banaya? A: Sir, yeh frontend development ka project hai. Aur SettleUp ka har kaam browser mein ho sakta hai — calculation, storage, sab kuch. Backend ki zaroorat tab padti jab data share karna ho ya multiple users ho, jo is version mein scope se bahar hai.

2.6 Assumptions and Dependencies

Assumption = jo maan ke chal rahe ho.

Example: "user ka browser modern hoga". Agar yeh galat nikla to app chalegi hi nahi.

Kyun likhte hain: Taaki baad mein pata rahe ki kaunsi cheez maan ke chale the. Agar app kisi ke system pe na chale to pehle assumptions check karo.

Dependency = jis cheez pe project depend karta hai.

SettleUp sirf browser pe depend karta hai. Koi library nahi, koi server nahi.

2.7 Project Scope Boundaries

Yeh section maine specially add kiya — isme features teen category mein baante hain:

Category	Matlab
Planned	Pakka banega. Nahi bana to project adhoora.
Recommended	Achha hoga agar ban jaye, lekin zaroori nahi
Future	Abhi nahi banega, aage ho sakta hai

Yeh kyun important hai: Agar SRS mein ek feature likha hai to woh **promise** hai. Teacher demo maangega. Isliye jo pakka nahi banana, usko clearly "Recommended" ya "Future" mark karna zaroori hai.

SECTION 3 — FUNCTIONAL REQUIREMENTS

Functional Requirement kya hota hai

Simple matlab: System ko **kya-kya kaam** karna chahiye.

Professional definition: A functional requirement specifies a behaviour or function that the system must perform.

Examples SettleUp se: - Member add kar sakega - Expense record kar sakega - Balance calculate karega - Settlement generate karega

Sab **kaam** hain. Actions hain.

Ab ek requirement ko line-by-line samajhte hain

SRS mein likha hai:

FR-11: The system shall compute the net balance of every member.

"FR-11" ka matlab

FR = Functional Requirement. 11 = gyarahvaan.

ID kyun dete hain: Taaki baat karte waqt reference de sakein. "FR-11 abhi implement nahi hua" bolna aasan hai, poori sentence dohrane se better. Aur testing mein bhi map kar sakte hain ki kaunsa test kis requirement ko check kar raha hai.

"shall" kyun likha, "will" ya "should" kyun nahi

Requirements writing mein teen shabd ka fixed matlab hai:

Shabd	Matlab
shall	Compulsory. Yeh hona hi chahiye.
should	Recommended. Achha hoga, lekin zaroori nahi.
may	Optional. Marzi hai.

Isliye har mandatory requirement mein "shall" likha hai. Yeh IEEE 830 ka convention hai.

Viva mein poocha ja sakta hai: "shall aur should mein kya farak hai?" — Jawab: shall matlab compulsory, should matlab optional recommendation.

"system" se kya matlab

"System" matlab SettleUp application. Requirement ye nahi keh rahi ki user calculate karega — **application** karegi, apne aap.

"net balance" kya hai

Ek member ne kitna paisa diya, minus uska kitna share banta tha.

Net Balance = Total Paid - Total Share

Positive = paisa milna hai. Negative = dena hai. Zero = hisaab barabar.

Yeh functional requirement kyun hai

Kyunki yeh ek **kaam** hai jo system karta hai. Agar likha hota "balance jaldi calculate hona chahiye" to woh non-functional hota — kyunki woh kaam nahi, **quality** hai.

Isko test kaise karenge

Manually calculate karo, phir app se compare karo.

Example: Amit ne ₹6,000 diya, teen logon mein baata. Amit ka share ₹2,000. Toh balance = $6000 - 2000 = +₹4,000$. App mein bhi +4,000 aana chahiye. (Yeh test case TC-17 hai.)

Viva Sawaal

Q: Functional requirement kya hota hai? A: Sir, functional requirement batata hai ki system ko kya kaam karna hai. Jaise "system member add kar sakega" ya "system balance calculate karega."

Q: FR-11 ko test kaise karoge? A: Manually ek example ka balance calculate karunga, phir app mein wahi expenses daal ke compare karunga. Dono same aane chahiye.

Q: Balance kaise calculate hota hai? A: Har member ka total paid nikalta hoon, phir uska total share nikalta hoon, aur ghata deta hoon. Positive aaye to usko paisa milna hai, negative aaye to dena hai.

Kuch aur important requirements

FR-02 — Unique member ID

| The system shall assign every member a unique numeric identifier.

Kyun zaroori hai: Do logon ka naam same ho sakta hai. Agar naam se pehchanenge to gadbad ho jayegi.

Yeh poorra explanation Section 5.4.1 mein hai — woh zaroor padhna, viva mein aata hai.

FR-08 — Equal split

| The system shall divide an expense equally among its participants.

Isme ek trap hai jo teacher pooch sakta hai:

₹100 ko teen logon mein baato to kya hoga?

$100 \div 3 = 33.333...$

Agar teenon ko ₹33.33 de do → total ₹99.99. **Ek paisa gayab.**

Isliye SRS mein likha hai ki bacha hua paisa distribute karna hai: - Pehle banda: ₹33.34 - Doosra: ₹33.33 - Teesra: ₹33.33 - **Total: ₹100.00 ✓**

Yeh kyun matter karta hai: Har expense mein thoda-thoda paisa gayab hoga, aur baad mein sab balances ka total zero nahi aayega. Phir settlement algorithm galat chalega, kyunki woh maanta hai ki total zero hai.

Viva Q: ₹100 ko 3 logon mein kaise baatoge? A: Sir, 33.34, 33.33 aur 33.33. Agar teenon ko 33.33 de doon to total 99.99 ho jayega aur ek paisa kam ho jayega. Isliye bacha hua paisa pehle bande ko de deta hoon.

FR-13 — Settlement generation

| The system shall generate a list of payments which, when made, reduces every member's net balance to zero.

Line by line:

- "list of payments" — kaun kisko kitna de, uski list
- "when made" — agar yeh saari payments ho jaayein
- "reduces every net balance to zero" — sabka hisaab barabar ho jayega

Test kaise karenge: Settlement generate karo, phir manually har payment apply karo balances pe. Sab zero aa jaane chahiye. (TC-24)

SECTION 4 — NON-FUNCTIONAL REQUIREMENTS

Non-Functional Requirement kya hota hai

Simple matlab: System **kaisa** hoga. Kaam nahi, **quality**.

Professional definition: A non-functional requirement specifies a quality attribute or constraint on how the system performs its functions.

Functional vs Non-Functional — ek hi example se

Ek hi cheez ko dono tarah likh ke dekho:

Type	Statement
Functional	"System settlement calculate karega"
Non-functional	"Settlement bina delay ke calculate hoga"

Pehla batata hai **kya** karega. Doosra batata hai **kaise** karega.

Yaad rakhne ka simple tareeka: - Functional = **verb** (kaam) — add karna, calculate karna, dikhana - Non-functional = **adjective** (quality) — fast, secure, easy, reliable

SettleUp ke non-functional requirements

Category	Kya kehta hai
Usability	Har field pe label ho, error saaf dikhe
Performance	Bina delay chale
Reliability	Data kharab ho to bhi app chale
Data Integrity	Balances ka total hamesha zero ho
Compatibility	Chrome, Firefox, Safari, Edge — sab pe chale

Category	Kya kehta hai
Privacy	Data kahin bahar na jaye
Maintainability	Code saaf ho, alag-alag files mein
Accessibility	Keyboard se bhi chal jaye

Ek important baat — numbers ke baare mein

Pehle SRS draft mein maine likha tha: "settlement 100 ms mein hona chahiye", "app 150 KB se kam ho".

Woh galat tha. Kyunki maine woh numbers kahin se naapa nahi tha, bas likh diya tha.

Viva mein risk: Teacher pooch sakta hai "100 ms hi kyun, 50 kyun nahi?" — aur jawab nahi hoga.

Isliye v2 mein numbers hata diye aur likha: "bina perceptible delay ke response dega, aur manual testing se verify hoga." Yeh **check bhi ho sakta hai** aur **jhooth bhi nahi hai**.

Viva Sawaal

Q: Non-functional requirement kya hota hai? A: Sir, yeh batata hai ki system kaam kaise karega — matlab quality. Jaise kitna fast hoga, kitna reliable, kaunse browsers pe chalega. Functional batata hai kya karega, non-functional batata hai kaisa hoga.

Q: Tumne performance ke liye koi number kyun nahi diya? A: Sir, maine abhi actual measurement nahi ki hai. Bina naape number likhna galat hota, kyunki uska koi base nahi hota. Isliye maine likha hai ki manual testing se verify karunga, aur agar development ke dauran number ki zaroorat pade to add kar dunga.

Q: Tumhara project secure hai? A: Nahi sir, aur maine SRS mein saaf likha hai. localStorage mein data plain text mein hota hai aur browser ke developer tools se koi bhi dekh sakta hai. Mera project **privacy** deta hai — matlab data kahin bahar nahi jaata — lekin **security** nahi deta. Yeh section 10.4 mein likha hai.

SECTION 5 — DATA REQUIREMENTS

Matlab

Data ka structure — kya-kya store hoga aur kis form mein.

SettleUp ka data

Member:

```
id    : number    (jaise 1, 2, 3)
name  : string    (jaise "Amit")
```

Expense:

```
id          : number
description : string    ("Hotel")
amount      : number    (6000)
paidBy      : number    (member ki id)
category    : string    ("Accommodation")
splitMethod : string    ("equal")
shares      : list      (kiska kitna hissa)
timestamp   : string    (kab record hua)
```

Sabse important part — ID vs Name

Yeh viva ka sabse pakka sawaal hai.

Problem kya hai

Maan lo do dost hain, dono ka naam "Rahul".

Ab agar expense mein likha hai paidBy: "Rahul" — to **kaunsa Rahul?** Pata hi nahi chalega.

Solution

Har member ko ek number do:

```
{ id: 1, name: "Rahul" }
{ id: 2, name: "Rahul" }    ← naam same, id alag
```

Aur expense mein likho:

```
{ paidBy: 1 }    ← ab clear hai
```

Teen wajah

1. Naam unique nahi hote. Do logon ka naam same ho sakta hai. Numbers hamesha unique rakhe ja sakte hain.

2. Naam badal sakte hain. Agar spelling galat thi aur baad mein theek ki, to naam se linked saare expenses toot jaayenge. ID se linked ho to naam badalne se kuch nahi hota.

3. Number compare karna aasan hai. Naam compare karne mein sochna padta hai — capital letters ka kya, extra space ka kya. Number mein yeh sab jhanjhat nahi.

Viva Sawaal

Q: Members ko ID se refer kyun kiya, naam se kyun nahi? A: Sir, teen wajah hain. Pehli — do logon ka naam same ho sakta hai, phir pata nahi chalega kisne pay kiya. Doosri — naam badla ja sakta hai, aur tab saare purane expenses ka reference toot jayega. Teesri — number compare karna simple hai, naam mein case aur space ka issue aata hai.

Q: Do members ka naam same ho to kya hoga? A: Sir, dono ki ID alag hogi, isliye system ko koi confusion nahi hogi. Expense mein ID store hoti hai, naam nahi. Bas display karte waqt naam dikhta hai.

Monetary Precision — DR-09

Problem: JavaScript mein $0.1 + 0.2$ ka jawab 0.3 nahi aata. Aata hai 0.30000000000000004 .

Kyun: JavaScript numbers ko binary floating point mein store karta hai, aur 0.1 jaisi decimal values binary mein exactly represent nahi hoti. Yeh JavaScript ki kami nahi, IEEE 754 standard ka behaviour hai — har language mein aisa hi hota hai.

Kyun matter karta hai: Bahut saare additions ke baad chhota-chhota error jud jaata hai. Phir balances ka total exactly zero nahi aata, aur settlement algorithm ki base assumption toot jaati hai.

Solution jo maine chuna: Sab calculation **paise mein** karna. ₹45.50 ko andar 4550 store karo. Integer arithmetic mein koi error nahi hota. Sirf display karte waqt wapass rupees banao.

Viva Sawaal

Q: JavaScript mein paise ka calculation karne mein kya problem hai? A: Sir, JavaScript floating point use karta hai, isliye $0.1 + 0.2$ exactly 0.3 nahi aata. Bahut saare calculations ke baad chhota error jud jaata hai aur balances ka total zero nahi rehta.

Q: Toh tumne kya solution kiya? A: Sab calculation paise mein karta hoon — integers mein. ₹45.50 ko 4550 bana ke store karta hoon. Integers mein koi rounding error nahi hoti. Display ke time wapass rupees mein convert kar deta hoon.

SECTION 6 — INTERFACE REQUIREMENTS

Matlab: Har screen pe user kya kar sakega.

Dhyaan dene wali baat: Isme yeh nahi likha ki button kis colour ka hoga ya kahan hoga. Sirf yeh likha hai ki **kya kar sakna chahiye**. Design implementation ke waqt decide hoga.

Kyun aisa likha: Agar SRS mein exact design likh dete aur baad mein badalna padta, to SRS galat ho jaata. Requirements design se zyada stable hoti hain.

Viva Sawaal

Q: SRS mein UI design kyun nahi diya? A: Sir, SRS batata hai ki system ko kya karna hai, kaise dikhna hai yeh design phase ka kaam hai. Maine likha hai ki har screen pe user kya kar sakega — jaise expense entry screen pe payer select kar sakega. Exact layout implementation ke waqt decide karunga.

SECTION 7 — ALGORITHM

Yeh sabse important section hai. Viva ke sabse zyada sawaal yahin se aayenge.

7.1 Problem kya hai

Group mein sab log alag-alag time pe alag-alag cheezein pay karte hain. Baad mein hisaab karna padta hai.

Naive tareeka: Har banda har uss bande ko de jise dena hai. Isse $n(n-1)/2$ tak payments ban sakti hain.

6 logon ke group mein = 15 payments. Har payment ek asli kaam hai jo kisi ko karna padta hai.

7.2 Net Balance

Net Balance = Total Paid - Total Share

Positive = milna hai. Negative = dena hai. Zero = barabar.

Ek property jo hamesha sach hai: Sab balances ka total **zero** hota hai.

Kyun: Jo paisa kisi ne diya, woh kisi na kisi ke share mein gaya. Total diya = total share. Farak zero.

Iska use: Agar zero nahi aaya to samajh jao kahin galti hai. Yeh ek **check** ban jaata hai.

7.4 Greedy Approach

Greedy algorithm kya hota hai: Har step pe jo sabse achha lag raha hai wahi choose karo, aur peeche mud ke mat dekho.

Yahan kya kar rahe hain: Sabse bade creditor ko sabse bade debtor se match karo, aur jitna ho sake utna settle kar do.

Kyun kaam karta hai: Jab dono mein se chhoti amount transfer karte ho, to kam se kam ek banda zero ho jaata hai aur nikal jaata hai. Har step pe ek banda kam. Isliye jaldi khatam ho jaata hai.

7.7 Worked Example — yeh yaad rakhna

Group: Amit, Riya, Karan

Expenses: - Amit ne hotel ka ₹6,000 diya, teenon mein barabar - Riya ne dinner ka ₹1,500 diya, teenon mein barabar

Har banda ka share: $(6000 + 1500) \div 3 = ₹2,500$

Balances:

Member	Diya	Share	Balance
Amit	₹6,000	₹2,500	+₹3,500
Riya	₹1,500	₹2,500	-₹1,000
Karan	₹0	₹2,500	-₹2,500

Check: $3500 - 1000 - 2500 = 0$ ✓

Settlement:

Step 1: Sabse bada creditor Amit (+3,500). Sabse bada debtor Karan (-2,500). Chhoti amount = 2,500. → **Karan pays Amit ₹2,500** Ab Amit +1,000, Karan 0 (nikal gaya)

Step 2: Amit (+1,000) aur Riya (-1,000). Chhoti amount = 1,000. → **Riya pays Amit ₹1,000** Dono zero. Khatam.

Result: 2 payments for 3 members. $n-1 = 2$. ✓

7.9 Transaction Count — YAHAN GALT MAT KARNA

Yeh sabse important cheez hai poore project mein.

Do alag cheezein hain jo log mix kar dete hain:

	Kya hai
Transaction count	Kitni payments karni padengi (log kitni baar paisa bhejenge)
Time complexity	Program kitna time lega chalne mein

SettleUp kya kam karta hai: Transaction count. $n(n-1)/2$ se **$n-1$** .

SettleUp ka time complexity: $O(n \log n)$ — sorting ki wajah se.

Yeh mat bolna

“Maine complexity $O(n^2)$ se $O(n)$ kar di”

Yeh **galat** hai. Complexity algorithm ke chalne ka time hai, payments ki sankhya nahi.

Yeh bolna

“Maine payments ki sankhya kam ki hai — $n(n-1)/2$ se $n-1$ tak. Algorithm ka time complexity $O(n \log n)$ hai kyunki balances sort karne padte hain.”

Yeh farak samajh ke bolna sabse zyada impress karega, kyunki zyadatar students yahi galti karte hain.

Comparison table — yeh yaad kar lo

Members	Naive: $n(n-1)/2$	SettleUp: $n-1$
3	3	2
4	6	3
5	10	4
6	15	5
10	45	9

7.10 Complexity

Step	Time
Creditors aur debtors alag karna	$O(n)$
Sorting	$O(n \log n)$
Main loop ($n-1$ baar)	$O(n)$
Total	$O(n \log n)$

Kyun $O(n \log n)$: Sorting sabse zyada time leti hai, isliye wahi total ban jaata hai.

Space: $O(n)$ — do lists aur payments ki list.

7.11 Limitation — imandari wala part

$n-1$ ek maximum guarantee hai, minimum nahi.

Matlab: SettleUp kabhi $n-1$ se zyada payments nahi banayega. Lekin kabhi-kabhi **theoretically usse bhi kam** possible ho sakti hain, aur greedy woh nahi dhoondh paata.

Kyun nahi dhoondte: Sabse kam payments nikalne ke liye balances ko aise groups mein baantna padta hai jinka har group ka total zero ho. Yeh **NP-hard** problem hai — matlab jaise-jaise members badhenge, computer ke liye solve karna namumkin hota jayega.

NP-hard ka simple matlab: Aisi problem jiska perfect answer bade inputs pe koi computer nikaal nahi sakta reasonable time mein.

Toh greedy kyun theek hai: $n-1$ already 15 se 5 wala bada improvement hai. Aur perfect answer se farak sirf ek-do payments ka hota hai, jo koi notice bhi nahi karega.

Viva Sawaal — yeh sab tayyari kar lena

Q: Greedy algorithm kya hota hai? A: Sir, greedy algorithm har step pe jo abhi sabse achha lag raha hai wahi choose karta hai, aur pichhla decision dobara nahi sochta. Mere case mein har step pe sabse bade creditor aur sabse bade debtor ko match karta hoon.

Q: Settlement algorithm kaise kaam karta hai? A: Pehle har member ka net balance nikalta hoon. Phir positive walon ko creditors aur negative walon ko debtors mein baant deta hoon, dono ko sort kar deta hoon. Phir sabse bade creditor aur sabse bade debtor ko lekar dono mein se chhoti amount transfer karta hoon. Isse kam se kam ek banda zero ho jaata hai aur nikal jaata hai. Yeh tab tak chalta hai jab tak dono lists khaali na ho jaayein.

Q: Payments $n-1$ hi kyun hoti hain, zyada kyun nahi? A: Sir, har step pe kam se kam ek banda zero hokar nikal jaata hai. Total n members hain. Jab ek hi bacha rahega to uska balance apne aap zero hoga, kyunki sabka total zero hota hai. Isliye maximum $n-1$ steps challenge, aur har step mein ek payment banti hai.

Q: Time complexity kya hai? A: $O(n \log n)$ sir. Sorting sabse zyada time leti hai. Loop sirf $n-1$ baar chalta hai jisme har baar constant kaam hota hai.

Q: Kya yeh sabse kam payments deta hai? A: Nahi sir, aur maine SRS mein yeh saaf likha hai. Yeh guarantee karta hai ki $n-1$ se zyada nahi hongi, lekin kabhi-kabhi theoretical minimum usse bhi kam ho sakta hai. Perfect minimum nikalna NP-hard problem hai, isliye greedy use kiya. Practical difference bahut kam hai.

Q: Greedy ke alawa kya use kar sakte the? A: Sir, small groups ke liye brute force se sab combinations check kar sakte hain aur perfect answer mil jayega. Lekin members badhne pe woh possible nahi rehta. Isliye greedy behtar choice hai.

SECTION 8 — BUSINESS RULES

Business Rule kya hota hai

Simple matlab: Aisa niyam jo software se pehle bhi sach hai. Tumne banaya nahi, woh pehle se exist karta hai.

Functional requirement se farak:

	Kehta hai
Functional requirement	"System yeh kaam karega"
Business rule	"Yeh hamesha sach rahega"

Example SettleUp se:

- FR: "System balance calculate karega" → kaam
- BR: "Sab balances ka total zero hoga" → niyam jo hamesha sach hai

Doosra wala accounting ka niyam hai. Maine banaya nahi, woh naturally sach hai.

SettleUp ke important business rules

ID	Rule
BR-05	Expense amount zero se zyada hona chahiye
BR-12	Exact shares ka total expense amount ke barabar ho
BR-14	Percentages ka total 100 ho
BR-17	Sab balances ka total zero ho
BR-20	Settlement mein zero ki payment na ho
BR-21	Settlement mein $n-1$ se zyada payments na ho

Viva Sawaal

Q: Business rule kya hota hai? A: Sir, business rule woh niyam hai jo problem domain mein pehle se sach hai, software se independent. Jaise "sab balances ka total zero hoga" — yeh accounting ka niyam hai, maine banaya nahi. Functional requirement batata hai system kya kaam karega, business rule batata hai kya hamesha sach rahega.

SECTION 9 — TESTING

Matlab

Kaise check karenge ki system sahi kaam kar raha hai.

SettleUp mein kaise test hoga

Manual testing. Koi automated framework nahi.

Kyun nahi: Testing framework use karna course ke scope se bahar hai, aur constraint C-03 kehta hai ki bina zaroorat library use nahi karni.

Isme jhooth nahi bolna. Pehle draft mein maine likha tha "automated tests shall cover the algorithm" — woh galat tha kyunki koi framework use hi nahi ho raha.

Test case kya hota hai

Teen cheezein: 1. **Precondition** — pehle kya state honi chahiye 2. **Action** — kya karna hai 3. **Expected result** — kya hona chahiye

Example (TC-11): - Precondition: teen members hain - Action: ₹100 ka expense equal split karo - Expected: shares ₹33.34, ₹33.33, ₹33.33 aur total ₹100.00

Edge case kya hota hai

Aisi ajeeb situation jahan code toot sakta hai.

SettleUp ke edge cases: - Group mein koi member hi na ho - Sirf ek member ho - Sab ka balance already zero ho - ₹100 teen logon mein baatna (divide nahi hota) - Ek banda jisne kisi expense mein hissa hi nahi liya

Kyun important: Normal case to sab chala lete hain. Edge cases hi asli test hote hain.

Viva Sawaal

Q: Testing kaise karoge? A: Manual testing sir, browser mein. Maine SRS mein 40 test cases likhe hain — har ek mein precondition, action aur expected result hai. Balances aur settlement check karne ke liye browser console bhi use karunga.

Q: Automated testing kyun nahi? A: Sir, uske liye framework chahiye hota hai jo project ke scope se bahar hai. Aur constraint bhi hai ki bina zaroorat koi library use na karein. Is size ke project ke liye manual testing kaafi hai.

Q: Edge case kya hota hai, example do? A: Edge case woh ajeeb situation hai jahan code toot sakta hai. Jaise ₹100 ko 3 logon mein baatna — divide exactly nahi hota, aur agar dhyan na do to ek paisa gayab ho jaata hai.

SECTION 10 — LIMITATIONS

Yeh section sabse zyada imandari dikhata hai. Isko halke mein mat lena.

localStorage ki limitations

Limitation	Matlab
Device-bound	Data sirf usi browser, usi device mein rehta hai
Clear ho sakta hai	Browsing data clear karo to data chala jaata hai
Private mode	Kuch browsers mein block ho jaata hai
Limit hoti hai	Har browser ki apni limit hai (exact number standard nahi hai)
Backup nahi	Automatic backup nahi hota, sirf manual export
Encrypted nahi	Plain text mein hota hai, koi bhi dekh sakta hai

Frontend-only ki limitations

- Multiple devices pe sync nahi hota
- Login nahi hai
- Data share nahi kar sakte
- Backup automatic nahi

Security ke baare mein — imandar rehna

Mera project secure NAHI hai. Aur yeh SRS mein saaf likha hai.

Privacy hai — matlab data kahin bahar nahi jaata. **Security nahi hai** — matlab device pe data protected nahi hai.

Do alag cheezein hain. Mix mat karna.

Viva Sawaal

Q: localStorage ki limitations kya hain? A: Sir, sabse badi yeh ki data sirf ek browser aur ek device mein rehta hai. Doosre device pe kuch nahi dikhega. Browsing data clear karne pe data chala jaata hai. Automatic backup nahi hota, aur data encrypted nahi hota. Yeh sab maine section 10.2 mein likha hai.

Q: Agar user browser data clear kar de to? A: Data chala jayega sir. Isliye maine JSON export ka feature rakha hai — user manually backup le sakta hai.

Q: Tumhare project ki sabse badi limitation kya hai? A: Sir, yeh ki data ek hi device pe rehta hai. Group ke doosre members apne phone pe nahi dekh sakte. Iske liye backend chahiye hota, jo is project ke scope se bahar hai. Maine isko future enhancement mein likha hai.

SECTION 11 — FUTURE ENHANCEMENTS

Matlab: Aage kya add ho sakta hai.

Kyun likhte hain: Dikhata hai ki aage ki soch rakhi hai. Aur clear ho jaata hai ki yeh abhi nahi bana.

Ek smart baat jo maine SRS mein daali hai: Har future enhancement ke saath likha hai ki **abhi wala kaunsa decision usse aasan banata hai.**

Jaise "rename member" — yeh isliye aasan hoga kyunki maine expenses mein ID store ki hai, naam nahi. Naam badalne se koi reference nahi tootega.

Viva mein yeh bolna: "Sir, maine data model aise banaya hai ki yeh future features aasani se add ho jaayein."

FINAL VIVA CHECKLIST

Yeh sab ke jawab pehle se soch ke rakhna:

#	Sawaal	Kahan se jawab
1	Yeh project kyun choose kiya?	Section 1.2 — trip ke baad hisaab karna real problem hai
2	Kya problem solve karta hai?	Manual calculation aur zyada payments
3	Vanilla JS kyun?	Section 2.5.1
4	localStorage kyun?	Section 2.5.1
5	Backend kyun nahi?	Section 2.5.1
6	ID vs naam?	Section 5.4.1 — do logon ka naam same ho sakta hai
7	Functional requirement kya hai?	Kaam — system kya karega
8	Non-functional kya hai?	Quality — system kaisa hoga
9	Scope vs objective?	Boundary vs purpose
10	Balance kaise nikalta hai?	Paid – Share
11	Creditor/debtor?	Positive = milna hai, negative = dena hai
12	Equal split kaise?	Divide karo, bacha hua paisa distribute karo

#	Sawaal	Kahan se jawab
13	₹100 teen mein?	33.34, 33.33, 33.33
14	Greedy algorithm kya?	Har step pe best choice, peeche nahi dekhna
15	Payments kam kaise hui?	Net balance se, individual debts se nahi
16	Time complexity?	$O(n \log n)$ — sorting ki wajah se
17	Greedy ki limitation?	Minimum guarantee nahi, sirf $n-1$ ka maximum
18	localStorage limitations?	Section 10.2
19	Invalid JSON import ho to?	Reject, purana data safe
20	Member delete ho jaye jiska expense hai?	Delete hi nahi hone denge

Aakhri salah

Ratna mat. Samajh ke jao. Agar teacher ne thoda alag tarike se pooch liya aur tumne ratta maara hai, to atak jaoge.

Sabse zyada tayyari Section 7 (algorithm) ki karna. Wahi tumhare project ka core hai aur wahi se sabse zyada sawaal aayenge.

Limitations wale sawaal se darna mat. "Yeh nahi kar sakta" bolna kamzori nahi hai — woh dikhata hai ki tumne apna system samjha hai. Jo student kehta hai "sir sab kuch kar sakta hai", uspe teacher ko shak hota hai.

Agar kuch nahi pata to bol dena. "Sir, yeh maine abhi socha nahi tha, main dekh ke bataunga" — yeh galat jawab banane se hazaar guna behtar hai.

Best of luck.